2025-2026

Welcome to the 14th Lab session of CO3519 module (Semester 02)

For Two Hours Lab Session:

#	Task	Time (m)	Guidance
A	Please perform the leftover from the previous lab sheet which is the **Task02** replacing the Digits dataset with CIFAR and reading CNN paper/understanding it.	55	This task is the usage of the same implementation which was used for Digit Classification. You just need to modify the code for CIFAR10 dataset. Also, perform the testing using different images with objects to see how your model performs.
	Break	05	
B	Please perform the **Task01** given on Page 2 for Applying Convolutional Neural Network and its Implementation for Facial Expression Recognition.	55	This task is the implementation of Facial expression recognition using CK+ dataset, please use the updated CK+ dataset found in this week folder. You can start from Page **2** to Page **7**. First you will train the model and then you will test the model. Next, I have uploaded an updated version of CK+ dataset, please download and use that.

For One Hour Lab Session:

#	Task	Time (m)	Guidance
1	Please perform the Task **B** from above two hours lab session with the given instructions	50	Follow the related guide in Task **B**.

I recommend downloading this PDF file so that you can view each line of code clearly.

Applying Convolutional Neural Network and its Implementation for Facial Expression Recognition

Important Note: Every student advised to carefully review each line and develop a clear understanding of the concepts. As we progress to more advanced and technical sessions in the coming weeks, it is essential to have a strong grasp of these foundational concepts. This will also support your preparation for the second assessment.

Task01: Implementation of CNNs for Facial Expression Recognition using CK+ dataset.

In the previous week, we implemented and discussed how simple CNN can be applied to solve the digit recognition problem. We used 10 classes of digits (0,1, …9) and understood how this problem worked. Now, we will extend the implementation of CNN for facial expression recognition for six classes which we used in the previous semester such as (Angry, Fear, Happy, Neutral, Sad, Surprise). Next, we will be firstly using **CK+ dataset** which is the most standard FER benchmarks. This dataset is already uploaded to the blackboard, and you already have used it. So simply move it to **My Drive** folder of You might get different accuracy depending how many epochs, layers, etc. you use. You can follow the step-by-step code with reading the given instruction for each code snippet.

Firstly, we will import the essential libraries where you will see tensorflow and keras for the first time. Some of the libraries are already familiar to you such as np, os, etc. but I will keep explaining them again & again to build your understanding about it.

NumPy (np): it helps with numerical operations and handling arrays.

OpenCV (cv2): This library is applied to do facial image processing (loading, editing, resizing, etc.) while OS **(os)** helps in handling the files and directories (like loading images from folders). Similarly, the **TensorFlow (tf) is a** machine learning framework for building neural networks. **Keras (tensorflow.keras)**: is used to import:

layers: Provide building blocks for deep learning models (e.g., convolutional layers, dense layers) and models: Used to define and compile a neural network. **Keras Preprocessing (image)**: Assists in handling image data before feeding it to the model.

```
import numpy as np
import cv2
import os
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing import image
```

This code is specific to Google Colab and is used to access files stored in Google Drive.

```
from google.colab import drive
drive.mount('/content/drive')
```

We already used this cascade for face detection in previous semester which is very common in detecting the faces in an image. We use it again to detect the faces before processing. In the below snippet, lets load it.

```
# Load Haar Cascade for face detection
face_cascade = cv2.CascadeClassifier("/content/drive/My Drive/haarcascade_frontalface_default.xml")
```

Once we load the cascade, lets prepare the data and load it. The below snippet loads and and preprocesses the dataset for expressions classification by detecting and extracting faces from images given in our dataset. It first sets a fixed input size of 48x48 pixels for consistency. The load_dataset function processes both training and testing data by iterating through six predefined emotion categories (anger, fear, happy, neutral, sadness, and surprise) each stored in separate subdirectories. Similarly, Within the load_images function, it checks if the corresponding folder exists before reading images using OpenCV (cv2.imread).

```
# Set image dimensions
img_height, img_width = 48, 48  # input size

def load_dataset(train_dir, test_dir):
    def load_images(directory):
        images = []
        labels = []
        expressions = ['anger', 'fear', 'happy', 'neutral', 'sadness', 'surprise']  # 6 emotion classes
        for label, expression in enumerate(expressions):
            expression_folder = os.path.join(directory, expression)
            if not os.path.exists(expression_folder):
                continue
            for filename in os.listdir(expression_folder):
                img_path = os.path.join(expression_folder, filename)
                img = cv2.imread(img_path)  # Read the image
                gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  # Convert to grayscale for face detection

                # Detect faces in the image
                faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))

                if len(faces) == 0:
                    continue

                x, y, w, h = faces[0]

                # Crop the face and resize to the desired size
                face_img = img[y:y+h, x:x+w]
                face_img = cv2.resize(face_img, (img_height, img_width))  # Resize to match network input size
                face_img = cv2.cvtColor(face_img, cv2.COLOR_BGR2GRAY)  # Convert the cropped face to grayscale
                face_img = np.expand_dims(face_img, axis=-1)  # Add channel dimension (grayscale -> shape (48, 48, 1)

                images.append(face_img)
                labels.append(label)

        images = np.array(images, dtype="float32") / 255.0  # Normalize the images to [0, 1]
        labels = np.array(labels)
        return images, labels

    # Load training and testing data
    X_train, y_train = load_images(train_dir)
    X_test, y_test = load_images(test_dir)

    return X_train, y_train, X_test, y_test

```

The images are converted to grayscale to improve face detection accuracy using a Haar cascade classifier (face_cascade.detectMultiScale). If no face is detected, the image is skipped; otherwise, the first detected face is cropped, resized to 48x48 pixels, and again converted to grayscale. To ensure compatibility with deep learning models, the image is reshaped to include a single-channel dimension ((48,48,1)). Once all images are processed, they are converted into NumPy arrays and normalized to

the [0,1] range by dividing pixel values by 255. Finally, the function returns the preprocessed images and corresponding labels for both training and testing sets, making them ready for model training.

Now, let's load the dataset, the paths I have given point to training and testing directories inside Google Drive because I set to use google collab. The dataset is stored in my Google Drive under "My Drive/CK_dataset/CK_dataset". Next, It processes the images, detects faces, resizes them, converts them to grayscale, normalizes pixel values, and returns:

X_train: Training images, y_train: Training labels, X_test: Testing images, y_test: Testing labels

```
train_dir = '/content/drive/My Drive/CK_dataset/CK_dataset/train'
test_dir = '/content/drive/My Drive/CK_dataset/CK_dataset/test'

# Load dataset
X_train, y_train, X_test, y_test = load_dataset(train_dir, test_dir)

# Check shape of the data to confirm it is correct
print("Training data shape:", X_train.shape)
print("Test data shape:", X_test.shape)
```

Now, we define our Convolutional Neural Network (CNN) model for classification using TensorFlow/Keras. The model is built using a sequential architecture, where layers are stacked one after another. It starts with a Conv2D layer with 32 filters of size (3,3), applying the ReLU activation function to capture features from the input images of shape (48,48,1). A MaxPooling2D layer (2x2) follows to reduce spatial dimensions and retain important features. This pattern repeats with increasing filters (64 and 128) in subsequent convolutional layers, enhancing the model's ability to detect complex patterns. In the previous week, you saw we didn't apply to use 128 filters for digits classification. Here, you will find the change as why we do so. Next, the Flatten layer converts the 2D feature maps into a 1D array for the dense layers. A fully connected Dense layer with 128 neurons and ReLU activation is added, followed by a Dropout layer (0.5 probability) to reduce overfitting. Finally, a Dense output layer with 6 neurons and a softmax activation function is used, corresponding to the 6 emotion classes, ensuring that the model outputs probability scores for classification.

```
# CNN Model
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(img_height, img_width, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(6, activation='softmax')  # 6 emotion classes
])
```

Now is the time to train and evaluate our model. This code compiles, trains, evaluates, and saves the CNN model where it compiles the model using the Adam optimizer for efficient gradient updates, sparse categorical cross-entropy as the loss function (since labels are integer-encoded), and accuracy as the evaluation metric. The model is then trained using the fit() function for 20 epochs with a batch size of 32, using X_train and y_train for training and X_test and y_test for validation. After training, the

model's performance is evaluated on the test dataset using evaluate(), printing the test accuracy as a percentage. Finally, the trained model is saved. In my case I saved it as an H5 file in my Google Drive, allowing it to be reloaded later without retraining. But you can use different path and use the same path while testing.

```
# Compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Train the model
model.fit(X_train, y_train, epochs=20, batch_size=32, validation_data=(X_test, y_test))

# Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test)
print(f"Test accuracy: {accuracy*100:.2f}%")

# Save the model
model.save('/content/drive/My Drive/face_expression_model.h5')  # Save as H5 model
```

Let's draw the confusion matrix. Here, you might see that fear classes have more wrong prediction. Why? Discuss it and think.

```
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Predict on the test set
y_pred = model.predict(X_test)
y_pred = np.argmax(y_pred, axis=1)  # Get class with highest probability

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

# Plot confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=['anger', 'fear', 'happy', 'neutral', 'sadness', 'surprise'],
            yticklabels=['anger', 'fear', 'happy', 'neutral', 'sadness', 'surprise'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```

Part of Task01: Single Image Test.

So, now we will load / deploy the trained model to test a single image. We will use the same kind of pre-processing, or any steps applied on the image in training stage, to the testing one. Please ensure, where have you saved your trained model and then update the model path accordingly. Next, I have added a folder under this week folder name 'FER-testingimages' with images to be considered for testing. After that, you uncomment the Line 28 - Line 31.

Next, I haven't added the details what each snippet does, I assume you already know about this as we did in previous lectures.

This code part is longer so please look at the indentation carefully and then proceed.

```python
import numpy as np
import cv2
import tensorflow as tf
import matplotlib.pyplot as plt  # For displaying images side by side

# Load the trained model
model = tf.keras.models.load_model('/content/drive/My Drive/face_expression_model.h5')

# Set image dimensions
img_height, img_width = 48, 48

# Dictionary of expression labels
expressions = ['anger', 'fear', 'happy', 'neutral', 'sadness', 'surprise']

# Load OpenCV's Haar cascade for face detection
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')

# Function to preprocess the face for prediction
def preprocess_face(face):
    face_resized = cv2.resize(face, (img_height, img_width))  # Resize
    face_resized = np.expand_dims(face_resized, axis=-1)  # Add channel dimension
    face_resized = np.expand_dims(face_resized, axis=0)  # Add batch dimension
    face_resized = face_resized.astype("float32") / 255.0  # Normalize
    return face_resized

# Path to the test image
img_path = '/content/drive/My Drive/FER-testingimages/0.jpg'
#img_path = '/content/drive/My Drive/FER-testingimages/faces-crop.jpg'
#img_path = '/content/drive/My Drive/FER-testingimages/test.png'
#img_path = '/content/drive/My Drive/FER-testingimages/test-2.png'
#img_path = '/content/drive/My Drive/FER-testingimages/test.png'

# Read the original image
original_img = cv2.imread(img_path)
gray_img = cv2.cvtColor(original_img, cv2.COLOR_BGR2GRAY)  # Convert to grayscale

# Detect faces in the image
faces = face_cascade.detectMultiScale(gray_img, scaleFactor=1.3, minNeighbors=5)

# Create copies of the image to draw bounding boxes
face_detected_img = original_img.copy()
final_img = original_img.copy()
```

```python

# Iterate through detected faces
for (x, y, w, h) in faces:
    face = gray_img[y:y+h, x:x+w]  # Extract face ROI
    processed_face = preprocess_face(face)  # Preprocess the face for the model

    # Get prediction
    prediction = model.predict(processed_face)
    predicted_class = np.argmax(prediction, axis=1)
    predicted_emotion = expressions[predicted_class[0]]

    # Draw bounding box on face detection image
    cv2.rectangle(face_detected_img, (x, y), (x + w, y + h), (255, 0, 0), 2)

    # Draw bounding box on final image
    cv2.rectangle(final_img, (x, y), (x + w, y + h), (0, 255, 0), 2)

    # Create a filled rectangle behind the text
    label_bg_color = (0, 255, 0)  # Green background
    text_color = (0, 0, 0)  # White text
    font = cv2.FONT_HERSHEY_SIMPLEX
    font_scale = 0.6
    font_thickness = 2

    # Get text size to draw background rectangle
    (text_width, text_height), _ = cv2.getTextSize(predicted_emotion, font, font_scale, font_thickness)
    label_x1, label_y1 = x, y  # Top-left corner of the bounding box
    label_x2, label_y2 = x + text_width + 10, y + text_height + 10  # Bottom-right corner for background

    # Draw the background rectangle
    cv2.rectangle(final_img, (label_x1, label_y1), (label_x2, label_y2), label_bg_color, -1)

    # Put the emotion label text inside the bounding box
    cv2.putText(final_img, predicted_emotion, (x + 5, y + text_height + 2), font,
                font_scale, text_color, font_thickness, cv2.LINE_AA)

# Convert BGR images to RGB for Matplotlib display
original_img_rgb = cv2.cvtColor(original_img, cv2.COLOR_BGR2RGB)
face_detected_img_rgb = cv2.cvtColor(face_detected_img, cv2.COLOR_BGR2RGB)
final_img_rgb = cv2.cvtColor(final_img, cv2.COLOR_BGR2RGB)

# Display all three images using matplotlib
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(original_img_rgb)
axes[0].set_title("Original Image")
axes[0].axis("off")

axes[1].imshow(face_detected_img_rgb)
axes[1].set_title("Image ")
axes[1].axis("off")

axes[2].imshow(final_img_rgb)
axes[2].set_title("Final Image with Emotion")
axes[2].axis("off")

plt.show()  # Show the images side by side
```

If you are very struggling with testing part of the code, I have added the link below where you will find the testing code. You can copy the testing part and make the necessary changes.

https://colab.research.google.com/drive/1_5Itef7nz3biLYyattUZgjgosMoUDKMV?usp=sharing